Software Requirements Specification (SRS)

College Club Management System (CCMS)

Document Version: 1.0.0
Status: Approved / Formal Specification
Standard: IEEE Std 830-1998 Compliant
Project: College Club Management System (CCMS)
Date: November 2026
Target Environment: Web (Vite + React 19 + TypeScript + Supabase PostgreSQL)

Table of Contents

1. Section 1: Introduction - 1.1 Purpose - 1.2 Scope of the System - 1.3 Project Objectives - 1.4 Intended Audience & Reading Suggestions
2. Section 2: Overall Description - 2.1 Product Perspective & Context - 2.2 User Classes & Characteristics - 2.3 Operating Environment - 2.4 Design & Implementation Constraints - 2.5 User Assumptions & Dependencies
3. Section 3: Functional Requirements - 3.1 Module 1: Authentication & Role-Based Access Control (RBAC) - 3.2 Module 2: Club & Category Directory Management - 3.3 Module 3: Membership Application & Review Workflow - 3.4 Module 4: Event Publishing & Scheduling - 3.5 Module 5: Event RSVP & Registration Management - 3.6 Module 6: Digital Attendance Tracking & Verification - 3.7 Module 7: Club Announcements & Broadcast System - 3.8 Module 8: Administrative Governance & Analytics
4. Section 4: Non-Functional Requirements - 4.1 Performance Requirements - 4.2 Security Requirements - 4.3 Usability Requirements - 4.4 Reliability & Availability Requirements - 4.5 Maintainability & Portability Requirements
5. Section 5: External Interface Requirements - 5.1 User Interfaces (UI) - 5.2 Hardware Interfaces - 5.3 Software Interfaces - 5.4 Communication & Protocol Interfaces
6. Section 6: Data Requirements - 6.1 Data Inputs & Capture Specifications - 6.2 Data Outputs & Presentation Specifications - 6.3 Storage & Volume Requirements - 6.4 Relational Database Specifications & Schema
7. Section 7: Use Cases - 7.1 Actors & System Boundary - 7.2 Detailed Use Case Descriptions (UC-01 through UC-08)
8. Section 8: Requirement Traceability - 8.1 Traceability Matrix (Requirements to Design & Test Cases)
9. Section 9: Document References & Supporting Information - 9.1 Applicable Standards - 9.2 Definitions, Acronyms, and Abbreviations

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete, formal, and unambiguous description of the **College Club Management System (CCMS)**. It outlines the functional capabilities, performance criteria, security constraints, external interface specifications, and architectural parameters required for the design, implementation, verification, and deployment of the CCMS platform. This document serves as the primary agreement between academic evaluators, stakeholders, development engineers, and quality assurance personnel for the IA3 Software Engineering Project evaluation.

1.2 Scope of the System

CCMS is a centralized, cloud-backed web platform engineered to modernize campus student engagement, eliminate paper-based workflows, and replace disorganized third-party messaging groups.

Core capabilities encompass: - Hardened role-based authentication separating **Students**, **Club Coordinators**, and **System Administrators**. - An interactive, searchable, and category-filtered catalog of campus clubs. - Transparent student membership application, review, and status tracking workflows. - Event publication, capacity-bounded RSVPs, and chronological scheduling. - Digital event attendance verification supporting institutional co-curricular activity records. - Club-wide official announcement broadcasting. - Administrative governance including club creation, coordinator assignment, and real-time visual analytics.

Out-of-Scope Capabilities: - Direct monetary payment processing for club dues or ticketed events (handled externally by college finance). - Third-party chat messaging or peer-to-peer social networking. - Physical biometric scanner hardware drivers (attendance is recorded digitally via coordinator dashboard interfaces).

1.3 Project Objectives

1. **Centralize Extracurricular Information:** Provide a single source of truth for all student clubs, categories, leadership contacts, and schedules.
2. **Automate Operational Lifecycles:** Eliminate manual paper forms for membership enrollment, event ticketing, and attendance tracking.
3. **Enforce Institutional Security & Integrity:** Implement strict PostgreSQL Row Level Security (RLS), eliminate privilege escalation vulnerabilities, and guarantee that public registration produces unprivileged student accounts only.
4. **Deliver Responsive, Accessible UX:** Provide a modern, high-contrast, accessible web interface adhering to WCAG 2.1 AA standards across desktop, tablet, and mobile browsers.

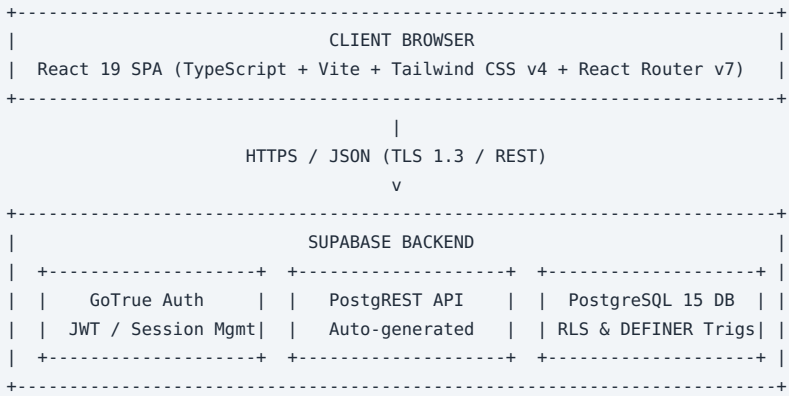
1.4 Intended Audience & Reading Suggestions

- **Academic Evaluators & Instructors:** Review Sections 1, 2, 7, and 8 to evaluate alignment with software engineering methodologies, use case modeling, and requirement traceability.
- **Frontend & Backend Developers:** Review Sections 3, 5, and 6 for granular implementation rules, API expectations, and relational database schemas.
- **Quality Assurance & Test Engineers:** Review Section 3 alongside Section 8 to verify test coverage and validate test cases against functional requirement identifiers.

2. Overall Description

2.1 Product Perspective & Context

CCMS operates as a self-contained, cloud-integrated web application. The architectural topology consists of a React 19 Single-Page Application (SPA) rendered on the client browser, communicating over TLS 1.3 with a Supabase cloud backend hosting a PostgreSQL 15+ relational database, GoTrue authentication engine, and PostgREST auto-generated RESTful API service.



2.2 User Classes & Characteristics

User Class	Technical Competence	Responsibilities & Access Rights
Student	Novice to Intermediate	Explores clubs, views public events, submits club membership requests, registers for events, views personal verified attendance, and receives announcements. Restricted to personal profile updates.
Club Coordinator	Intermediate	Manages assigned club(s), reviews student membership applications (Approve/ Reject), drafts and publishes events, takes and edits event attendance rosters, and broadcasts club announcements.
System Administrator	Advanced	Has global authority across the entire platform. Creates clubs, defines club categories, appoints/removes student coordinators, oversees system metrics, and manages user accounts.

2.3 Operating Environment

- **Client Platforms:** Modern evergreen web browsers (Chromium >= 120, Firefox >= 122, Safari >= 17, Edge >= 120).
- **Client Form Factors:** Fully responsive across mobile (>= 360px), tablet (>= 768px), laptop (>= 1024px), and desktop (>= 1440px).
- **Backend Infrastructure:** Supabase Cloud / PostgreSQL 15.6, Node.js 20+ runtime for local build and testing.

2.4 Design & Implementation Constraints

1. **Security Isolation:** Role enforcement must execute at the database tier via PostgreSQL Row Level Security (RLS) and `SECURITY DEFINER` trigger functions. Client-side state must never be trusted for access control decisions.
2. **Deterministic Role Creation:** The `handle_new_user()` database trigger must unconditionally force `role = 'student'` upon signup, discarding any client-provided metadata.
3. **Database Portability:** Code must adhere to standard ANSI SQL and PostgreSQL PL/pgSQL without proprietary extensions beyond standard cryptographic UUID generators (`gen_random_uuid()`).
4. **Modern UI Framework:** Built strictly using React 19, TypeScript strict mode (`strict: true`), Vite 8, and Tailwind CSS v4.

2.5 User Assumptions & Dependencies

- Users possess an active internet connection and a valid email address.
- The university provides institutional email addresses for identity verification.
- Supabase cloud service maintains a minimum of 99.9% uptime for authentication and database services.

---

3. Functional Requirements

3.1 Module 1: Authentication & Role-Based Access Control (RBAC)

- **FR-AUTH-01 (Student Self-Registration):** The system shall allow unauthenticated visitors to register an account using full name, email, and password. The system must unconditionally assign the role `'student'` regardless of client parameters.
- **FR-AUTH-02 (User Authentication):** The system shall authenticate registered users using email and password credentials, returning a secure JWT and session token upon successful validation.
- **FR-AUTH-03 (Role-Based Route Protection):** The client router shall inspect user roles and redirect authenticated users to their corresponding dashboard:
- Students $\rightarrow$ `/student/dashboard`
- Coordinators $\rightarrow$ `/coordinator/dashboard`
- Administrators $\rightarrow$ `/admin/dashboard`
- **FR-AUTH-04 (Role Tamper Protection):** The database trigger `protect_profile_role` shall intercept all `UPDATE` statements on `public.profiles` and reject any modification to the `role` column unless the executing identity is an authorized administrator (`public.is_admin() = true`) or authenticated through the atomic bootstrap procedure.
- **FR-AUTH-05 (First-Admin Bootstrap):** The system shall provide a dedicated procedure `public.bootstrap_first_admin(uuid)` allowing the database administrator to promote the first user to `'admin'` using a transaction-level advisory lock (`pg_advisory_xact_lock`) and a private token table (`private_ccms.admin_bootstrap_tokens`), rejecting execution once an administrator exists.
- **FR-AUTH-06 (Session Invalidation):** The system shall enable users to securely sign out from any authenticated screen, purging active tokens from browser storage.

3.2 Module 2: Club & Category Directory Management

- **FR-CLUB-01 (Public Club Directory):** The system shall display an interactive catalog of all active clubs, displaying club name, category, description, website, and coordinator contact.
- **FR-CLUB-02 (Category Taxonomy):** The system shall categorize clubs into predefined domains (e.g., Academic, Cultural, Technical, Sports) and allow real-time client-side category filtering.

- **FR-CLUB-03 (Club Search):** The system shall provide an instantaneous, case-insensitive text search filtering clubs by name and description.
- **FR-CLUB-04 (Club Profile Page):** The system shall generate a dedicated club details page for each club via a human-readable URL slug (`/clubs/:slug`), presenting upcoming events, announcements, and an application trigger.
- **FR-CLUB-05 (Admin Club Creation):** The system shall restrict club creation and club deletion strictly to users with the `'admin'` role.
- **FR-CLUB-06 (Coordinator Club Editing):** The system shall permit assigned coordinators to edit their specific club's description, contact email, and banner URL via an authenticated dashboard modal.

3.3 Module 3: Membership Application & Review Workflow

- **FR-MEM-01 (Membership Application Submission):** An authenticated student shall be able to submit a membership request to any club with an initial lifecycle status of `'pending'`.
- **FR-MEM-02 (Duplicate Application Prevention):** The database shall enforce a unique composite constraint `(club_id, student_id)` on `public.club_memberships`, preventing duplicate applications.
- **FR-MEM-03 (Membership Immutable Fields Trigger):** The trigger `protect_club_membership_immutable_fields` shall prohibit any update that alters `student_id` or `club_id` on an existing membership record.
- **FR-MEM-04 (Coordinator Application Review):** Club coordinators shall view a roster of pending applications for their club and execute either an `'approved'` or `'rejected'` status transition.
- **FR-MEM-05 (Membership Cancellation):** A student shall be permitted to withdraw/cancel their own membership application provided the status is `'pending'`.

3.4 Module 4: Event Publishing & Scheduling

- **FR-EVT-01 (Event Creation):** Authorized coordinators and administrators shall create events specifying title, description, club affiliation, event date, start time, venue/location, and maximum capacity limit.
- **FR-EVT-02 (Public Event Visibility):** All active campus events shall be publicly visible to all users and categorized into "Upcoming Events" and "Past Events" relative to the current timestamp.
- **FR-EVT-03 (Event Modification):** Coordinators shall have update authority strictly over events hosted by their assigned club (`public.is_coordinator_for_club(club_id) = true`).
- **FR-EVT-04 (Event Deletion):** Coordinators and administrators shall have permission to delete events, which automatically cascades to remove associated registrations and attendance records.

3.5 Module 5: Event RSVP & Registration Management

- **FR-REG-01 (One-Click Student Registration):** Authenticated students shall be able to register for any upcoming event with a single click.
- **FR-REG-02 (Duplicate Registration Guard):** The system shall prevent a student from registering multiple times for the same event via unique constraint `(event_id, student_id)`.
- **FR-REG-03 (Capacity Limit Enforcement):** The registration service shall query the current count of active registrations and reject registration if `current_count >= event.capacity`.
- **FR-REG-04 (Registration Cancellation):** Students shall be permitted to unregister from an event up to the event start time, freeing capacity for other students.
- **FR-REG-05 (Registration Roster Visibility):** Coordinators shall view the real-time list of all students registered for events organized by their club.

3.6 Module 6: Digital Attendance Tracking & Verification

- **FR-ATT-01 (Attendance Roster Generation):** When an event occurs, coordinators shall access an attendance management interface populated with all registered students.
- **FR-ATT-02 (Status Recording):** Coordinators shall record the attendance status of each attendee as one of: `'present'`, `'absent'`, or `'late'`.
- **FR-ATT-03 (Attendance Modification):** Coordinators shall be able to update attendance records within 48 hours following event conclusion to rectify mistakes.
- **FR-ATT-04 (Student Attendance Transcript):** Students shall access an attendance summary on their personal dashboard displaying their total attended events, verified attendance percentage, and event history badges.
- **FR-ATT-05 (Attendance Security):** Direct insertion or modification of attendance records by students shall be rejected by PostgreSQL RLS policies.

3.7 Module 7: Club Announcements & Broadcast System

- **FR-ANN-01 (Announcement Authoring):** Coordinators shall compose announcements with title, content body, and target club ID.
- **FR-ANN-02 (Public Announcement Feed):** All announcements shall be displayed on the club details page in reverse chronological order.
- **FR-ANN-03 (Student Notification Feed):** Students shall view recent announcements from clubs in which they hold `'approved'` membership directly on their student dashboard.
- **FR-ANN-04 (Announcement Deletion):** Announcements may be removed by the authoring coordinator or system administrators.

3.8 Module 8: Administrative Governance & Analytics

- **FR-ADM-01 (Club Coordinator Assignment):** Administrators shall assign registered students as coordinators of specific clubs by inserting records into `public.club_coordinators`.
- **FR-ADM-02 (Automatic Role Elevation):** The trigger `sync_coordinator_role_on_assign` shall automatically elevate a student's profile role to `'coordinator'` upon assignment.
- **FR-ADM-03 (Automatic Role Demotion):** The trigger `sync_coordinator_role_on_unassign` shall revert a coordinator's profile role to `'student'` upon removal if they hold no other coordinator assignments.
- **FR-ADM-04 (Category CRUD):** Administrators shall create, edit, and delete club categories.
- **FR-ADM-05 (Institutional Analytics):** The admin dashboard shall render interactive visual analytics (using Recharts) displaying club distribution by category, active student registration trends, and attendance rates.
- **FR-ADM-06 (User Management):** Administrators shall inspect all user profiles, filter by role, and revoke privileges when required.

---

4. Non-Functional Requirements

4.1 Performance Requirements

- **NFR-PERF-01 (Page Load Time):** First Contentful Paint (FCP) shall occur within $\le 1.2\text{ seconds}$ and Largest Contentful Paint (LCP) within $\le 2.0\text{ seconds}$ under standard 4G broadband connections.
- **NFR-PERF-02 (API Latency):** Read queries against cached or indexed tables (e.g., club directory, announcements) shall resolve within $\le 250\text{ ms}$ at 95th percentile.

- **NFR-PERF-03 (Concurrent Operations):** The system shall handle at least 500 concurrent active student sessions during peak campus registration events without session drops.
- **NFR-PERF-04 (Client Bundle Optimization):** The minified and gzipped JavaScript production bundle shall remain $\le 200\text{ kB}$ for core vendor chunks through route-level code splitting.

4.2 Security Requirements

- **NFR-SEC-01 (Database-Tier RLS):** 100% of public database tables (all 9 tables) must enforce Row Level Security (`ENABLE ROW LEVEL SECURITY`). Zero application data shall be accessible via default public permissions.
- **NFR-SEC-02 (Search Path Hardening):** Every `SECURITY DEFINER` function must explicitly configure `SET search_path = ''` and reference all database objects using fully qualified schema identifiers to eliminate search-path injection vulnerabilities.
- **NFR-SEC-03 (Client Role Sanitization):** Frontend builds must only reference `VITE_SUPABASE_URL` and `VITE_SUPABASE_PUBLISHABLE_KEY`. Service role keys (`SUPABASE_SERVICE_KEY`) must never be bundled into client-side code.
- **NFR-SEC-04 (Password Cryptography):** User passwords must be hashed using bcrypt or Argon2id with a work factor ≥ 10, handled natively by the GoTrue cryptographic identity provider.
- **NFR-SEC-05 (Data in Transit):** All client-server communications must be encrypted using Transport Layer Security (TLS 1.3), rejecting unencrypted HTTP.
- **NFR-SEC-06 (Input Sanitization & SQL Injection Defense):** All database interactions must use parameterized statements generated by the Supabase client library. Direct dynamic SQL string concatenation is strictly prohibited.

4.3 Usability Requirements

- **NFR-USE-01 (Visual Contrast Standards):** All screen elements, typography, and interactive components must satisfy WCAG 2.1 Level AA contrast ratios (minimum 4.5:1 for body copy and 3.0:1 for large display headings).
- **NFR-USE-02 (Intuitive Navigation):** Common student workflows (such as exploring clubs or registering for an event) must be achievable in $\le 3\text{ user interactions (clicks)}$.
- **NFR-USE-03 (Responsive Layout Stability):** Cumulative Layout Shift (CLS) score shall be ≤ 0.05 across all responsive viewport breakpoints.
- **NFR-USE-04 (Actionable Error Feedback):** All form validation errors (e.g., invalid email format, weak password, duplicate registration) must display user-friendly contextual messages inline within $\le 100\text{ ms}$.

4.4 Reliability & Availability Requirements

- **NFR-REL-01 (System Uptime):** The platform shall deliver a minimum service availability of 99.9% uptime during the academic semester, excluding planned maintenance windows.
- **NFR-REL-02 (Fault Tolerance & Session Preservation):** In the event of temporary client network disconnection, active user input within modal forms must be preserved in component state.
- **NFR-REL-03 (Transaction Integrity):** All multi-table updates (such as coordinator role assignments or first-admin bootstrapping) must execute inside atomic ACID transactions, rolling back completely on any exception.

4.5 Maintainability & Portability Requirements

- **NFR-MAINT-01 (Modular Architecture):** The codebase shall follow a strict modular layered structure (`/components`, `/contexts`, `/pages`, `/services`, `/types`) with explicit TypeScript interfaces.

- **NFR-MAINT-02 (Zero Lint Defects):** Code must compile with 0 TypeScript compilation errors (`tsc -b`) and 0 linter violations (`oxlint` / `eslint`).
- **NFR-MAINT-03 (Database Migrations):** All database schema changes must be captured in idempotent, version-controlled SQL migration scripts under `supabase/migrations/`.

---

5. External Interface Requirements

5.1 User Interfaces (UI)

The CCMS client UI is developed using Tailwind CSS v4 and the JetBrains Mono / Inter typography system.

1. **Landing Page (`/`):** - High-contrast hero section with animated background gradients. - Live campus registration ticker banner with status indicator. - Interactive domain filter chips for instantaneous discovery. - Interactive Live Campus Showcase Card demonstrating real-time capacity and attendance counters. - Responsive navbar and accessible footer.
2. **Clubs Catalog (`/clubs`):** - Filterable, searchable card grid with real-time text input and category dropdown. - Responsive card layouts showing club badges, category tags, and slug links.
3. **Club Details View (`/clubs/:slug`):** - Club banner, description, coordinator contact info, and dynamic membership application button. - Tabs for "Upcoming Events" with registration triggers and "Announcements" feed.
4. **Student Portal (`/student/dashboard`):** - Overview metrics: Joined clubs, registered events, overall attendance rate. - My Clubs list with status indicators (`pending`, `approved`, `rejected`). - Upcoming registered events timeline with calendar timestamps. - Verified attendance history card.
5. **Coordinator Portal (`/coordinator/dashboard`):** - Club management tools, pending membership review approval/rejection buttons. - Event creation modal form with date-pickers and capacity caps. - Attendance marking grid with `Present`, `Absent`, and `Late` toggles. - Announcement drafting and broadcast interface.
6. **Administrator Console (`/admin/dashboard`):** - Campus-wide metrics overview and Recharts analytics. - Club CRUD modal with coordinator assignment selectors. - Category creation and deletion table. - Complete student directory with role modification controls.

5.2 Hardware Interfaces

CCMS requires no specialized client hardware. The application operates on standard computing hardware: - **Processor:** Any modern dual-core x86-64 or ARM processor. - **Memory:** Minimum 2 GB RAM available to client operating system. - **Display Resolution:** Minimum resolution $360 \times 640\text{ px}$; recommended resolution $1920 \times 1080\text{ px}$.

5.3 Software Interfaces

- **Operating Systems:** Windows 10/11, macOS 12+, Linux (Ubuntu/Debian/Fedora), Android 11+, iOS 15+.
- **Supabase JavaScript SDK (`@supabase/supabase-js` v2.117+):** Provides WebSocket connections for real-time events and HTTPS REST bindings to PostgREST.
- **Vite Build System:** Orchestrates Hot Module Replacement (HMR) and optimized rollup production bundles.
- **Recharts Library:** Renders SVG-based data visualizations for administrative reports.

5.4 Communication & Protocol Interfaces

- **HTTPS:** Port 443 with TLS 1.3 encryption for all client-to-backend REST API calls.

- **WSS (Secure WebSockets):** Port 443 for real-time Supabase database change subscriptions.
- **JSON:** All request payloads and response bodies adhere strictly to `application/json` formatting.

---

6. Data Requirements

6.1 Data Inputs & Capture Specifications

- **User Registration:** Full Name (2–100 chars), Email (valid institutional RFC 5322 format), Password (minimum 6 characters).
- **Club Creation:** Name (3–100 chars), Slug (URL-safe string, unique), Description (text), Category ID (foreign key UUID), Contact Email (valid email format).
- **Event Creation:** Title (3–150 chars), Club ID (foreign key UUID), Event Date (ISO 8601 date, $\ge \text{today}$), Start Time (time string), Location (text), Capacity Limit (positive integer > 0).
- **Attendance Record:** Event ID (foreign key UUID), Student ID (foreign key UUID), Status (`'present'`, `'absent'`, or `'late'`), Marked At (timestamptz).

6.2 Data Outputs & Presentation Specifications

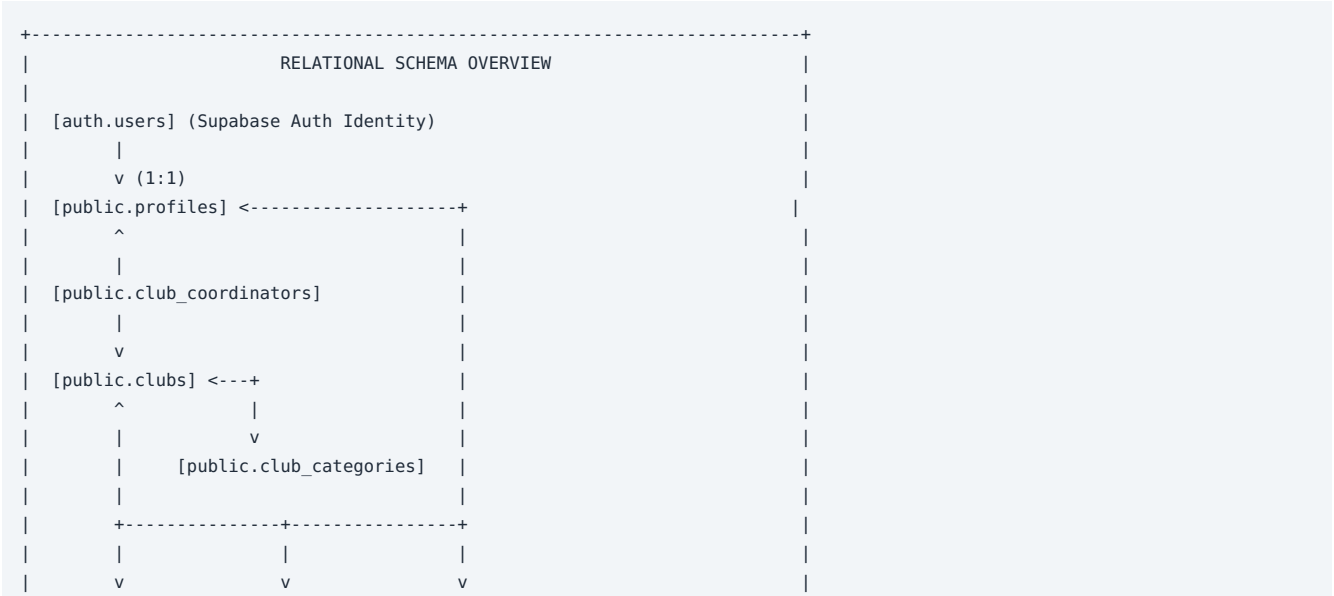
- **Event Capacity Gauge:** Computed integer ratio displaying `Registered Count / Maximum Capacity` and remaining seats.
- **Attendance Rate Metric:** Percentage value computed as: $$\text{Attendance Rate} = \left(\frac{\text{Count of 'present' records}}{\text{Total registered event sessions}} \right) \times 100\%$$
- **Timestamp Displays:** Formatted using client locale conventions (e.g., `Nov 15, 2026, 10:00 AM`).

6.3 Storage & Volume Requirements

- The database shall support storage for at least 10,000 active student profiles, 100 campus clubs, 2,000 annual events, and 50,000 attendance records with total storage consumption under 2 GB.
- Static assets (club logos, banners) shall be stored via external CDN URLs or Supabase Storage buckets.

6.4 Relational Database Specifications & Schema

The database consists of 9 public relational tables and 1 private security token table:



```
|  [public.events] [public.announcements] [public.club_memberships]        |
|       |                                                                   |
|       +-------------------------------+                                   |
|       |                               |                                   |
|       v                               v                                   |
|  [public.event_registrations]    [public.attendance]                      |
+---------------------------------------------------------------------------+
```

Table Specifications:

1. **`public.profiles`** - `id` (uuid, Primary Key, references `auth.users.id` on delete cascade) - `full_name` (text, not null) - `email` (text, not null) - `role` (text, not null, check `role in ('student', 'coordinator', 'admin')`, default `'student'`) - `student_id` (text, nullable) - `department` (text, nullable) - `year` (text, nullable) - `phone` (text, nullable) - `avatar_url` (text, nullable) - `created_at` (timestamptz, default `now()`) - `updated_at` (timestamptz, default `now()`)
2. **`public.club_categories`** - `id` (uuid, Primary Key, default `gen_random_uuid()`) - `name` (text, not null, unique) - `description` (text, nullable) - `created_at` (timestamptz, default `now()`)
3. **`public.clubs`** - `id` (uuid, Primary Key, default `gen_random_uuid()`) - `name` (text, not null) - `slug` (text, not null, unique) - `description` (text, nullable) - `category_id` (uuid, references `club_categories.id` on delete set null) - `contact_email` (text, nullable) - `website_url` (text, nullable) - `banner_url` (text, nullable) - `created_at` (timestamptz, default `now()`) - `updated_at` (timestamptz, default `now()`)
4. **`public.club_coordinators`** - `id` (uuid, Primary Key, default `gen_random_uuid()`) - `club_id` (uuid, not null, references `clubs.id` on delete cascade) - `coordinator_id` (uuid, not null, references `profiles.id` on delete cascade) - `created_at` (timestamptz, default `now()`) - Unique constraint: `(club_id, coordinator_id)`
5. **`public.club_memberships`** - `id` (uuid, Primary Key, default `gen_random_uuid()`) - `club_id` (uuid, not null, references `clubs.id` on delete cascade) - `student_id` (uuid, not null, references `profiles.id` on delete cascade) - `status` (text, not null, check `status in ('pending', 'approved', 'rejected')`, default `'pending'`) - `joined_at` (timestamptz, default `now()`) - Unique constraint: `(club_id, student_id)`
6. **`public.events`** - `id` (uuid, Primary Key, default `gen_random_uuid()`) - `club_id` (uuid, not null, references `clubs.id` on delete cascade) - `title` (text, not null) - `description` (text, nullable) - `event_date` (date, not null) - `start_time` (time, not null) - `location` (text, not null) - `capacity` (integer, not null, default 100) - `created_at` (timestamptz, default `now()`)
7. **`public.event_registrations`** - `id` (uuid, Primary Key, default `gen_random_uuid()`) - `event_id` (uuid, not null, references `events.id` on delete cascade) - `student_id` (uuid, not null, references `profiles.id` on delete cascade) - `registered_at` (timestamptz, default `now()`) - Unique constraint: `(event_id, student_id)`
8. **`public.attendance`** - `id` (uuid, Primary Key, default `gen_random_uuid()`) - `event_id` (uuid, not null, references `events.id` on delete cascade) - `student_id` (uuid, not null, references `profiles.id` on delete cascade) - `status` (text, not null, check `status in ('present', 'absent', 'late')`) - `marked_at` (timestamptz, default `now()`) - Unique constraint: `(event_id, student_id)`
9. **`public.announcements`** - `id` (uuid, Primary Key, default `gen_random_uuid()`) - `club_id` (uuid, not null, references `clubs.id` on delete cascade) - `title` (text, not null) - `content` (text, not null) - `created_at` (timestamptz, default `now()`)
10. **`private_ccms.admin_bootstrap_tokens`**
    - `target_user_id` (uuid, Primary Key)
    - `backend_pid` (integer, not null)
    - `created_at` (timestamptz)

- `is_singleton` (boolean, not null, check `is_singleton = true`, unique)

7. Use Cases

7.1 Actors & System Boundary

- **Guest / Unregistered Visitor:** Discovers clubs, reads announcements.
- **Student Actor:** Registers, joins clubs, RSVPs for events, tracks attendance.
- **Coordinator Actor:** Approves memberships, schedules events, marks attendance, posts news.
- **Admin Actor:** Governs clubs, appoints leadership, views system analytics.

7.2 Detailed Use Case Descriptions

```
+---------------------------------------------------------------------------+
|                          USE CASE SPECIFICATION                           |
|                                                                           |
|  [Guest] ------> (UC-01: User Self-Registration)                          |
|  [Student] ----> (UC-02: Search & Apply for Club Membership)               |
|  [Student] ----> (UC-03: Event RSVP with Capacity Guard)                  |
|  [Student] ----> (UC-04: View Personal Verified Attendance)               |
|  [Coord] ------> (UC-05: Review Club Membership Application)              |
|  [Coord] ------> (UC-06: Mark & Verify Event Attendance)                  |
|  [Admin] ------> (UC-07: Create Club & Assign Coordinator)                |
|  [DB Owner] ---> (UC-08: First Administrator Bootstrap)                   |
+---------------------------------------------------------------------------+
```

UC-01: User Self-Registration

- **Actor:** Guest / Student
- **Preconditions:** User has not logged in and possesses a valid email.
- **Trigger:** Clicks "Get Started" on the landing page or navigates to `/register`.
- **Primary Flow:** 1. User enters Full Name, Email, and Password. 2. User submits registration form. 3. Client dispatches `supabase.auth.signUp()`. 4. Database trigger `handle_new_user()` executes with `SECURITY DEFINER`. 5. Profile row created in `public.profiles` with `role = 'student'` unconditionally. 6. User is redirected to `/login` or logged in to `/student/dashboard`.
- **Alternative Flow (Email Exists):**
- Database returns unique constraint violation; UI displays "User already registered" message.
- **Postconditions:** User identity exists in `auth.users` and associated `'student'` profile in `public.profiles`.

UC-02: Search & Apply for Club Membership

- **Actor:** Authenticated Student
- **Preconditions:** Student is logged in; viewing `/clubs`.
- **Primary Flow:** 1. Student enters search keywords or clicks category filter chips. 2. System filters matching clubs instantaneously in real-time. 3. Student selects a club card to view details (`/clubs/:slug`). 4. Student clicks "Join Club". 5. Client calls `supabase.from('club_memberships').insert(...)`. 6. Row inserted with `status = 'pending'`. 7. UI updates button state to "Membership Pending".
- **Alternative Flow (Already Applied):**
- If a membership record exists, UI displays "Membership Pending" or "Already a Member" and disables the button.

- **Postconditions:** Record created in `public.club_memberships` with status `'pending'`.

UC-03: Event RSVP with Capacity Guard

- **Actor:** Authenticated Student
- **Preconditions:** Student logged in; viewing upcoming event on club details page.
- **Primary Flow:** 1. Student views event details and remaining capacity gauge. 2. Student clicks "Register for Event". 3. System checks that current registrations are less than event capacity. 4. System inserts record into `public.event_registrations`. 5. UI displays confirmed registration badge and decrements remaining seat counter.
- **Alternative Flow (Event Full):**
- If registration count equals capacity, registration button is disabled with text "Event Full".
- **Postconditions:** Registration saved in database; capacity updated in real time.

UC-04: View Personal Verified Attendance

- **Actor:** Authenticated Student
- **Preconditions:** Student has attended one or more events.
- **Primary Flow:** 1. Student navigates to `/student/dashboard`. 2. System queries `public.attendance` where `student_id = auth.uid()`. 3. System calculates overall attendance percentage. 4. UI renders the Attendance Summary Card with attendance percentage and chronological event badges (`Present`, `Absent`, `Late`).
- **Postconditions:** Attendance transcript displayed without allowing client-side modifications.

UC-05: Review Club Membership Application

- **Actor:** Club Coordinator
- **Preconditions:** Coordinator logged in and assigned to the club.
- **Primary Flow:** 1. Coordinator opens `/coordinator/dashboard`. 2. System loads pending applications for coordinator's club. 3. Coordinator reviews student name, email, and submission date. 4. Coordinator clicks "Approve" (or "Reject"). 5. Database updates `public.club_memberships.status` to `'approved'` (or `'rejected'`). 6. Trigger `protect_club_membership_immutable_fields` validates that `student_id` and `club_id` were not altered. 7. Roster updates in real time.
- **Postconditions:** Student membership status transitioned to `'approved'` or `'rejected'`.

UC-06: Mark & Verify Event Attendance

- **Actor:** Club Coordinator
- **Preconditions:** Event has concluded or is actively occurring.
- **Primary Flow:** 1. Coordinator selects an event from the "Attendance" tab in `/coordinator/dashboard`. 2. System queries `public.event_registrations` for that event and displays registered students. 3. Coordinator toggles status buttons: `Present`, `Absent`, or `Late` for each student. 4. System upserts rows into `public.attendance`. 5. Visual confirmation badge displayed for each updated student.
- **Postconditions:** Official institutional attendance records created and visible to students.

UC-07: Create Club & Assign Coordinator

- **Actor:** System Administrator
- **Preconditions:** Administrator logged in with `role = 'admin'`.
- **Primary Flow:** 1. Administrator navigates to `/admin/dashboard`. 2. Administrator clicks "Create Club", fills in Name, Category, Description, and Slug. 3. System inserts new record into `public.clubs`. 4. Administrator selects a registered student and assigns them as club coordinator. 5. System inserts row

into `public.club_coordinators`. 6. Database trigger `sync_coordinator_role_on_assign` promotes student profile to `'coordinator'`.

- **Postconditions:** Club published live; assigned student promoted to coordinator.

UC-08: First Administrator Bootstrap

- **Actor:** Database Administrator / Database Owner (`postgres`)
- **Preconditions:** Fresh database migration deployed; 0 administrators exist in `public.profiles`.
- **Primary Flow:** 1. Administrator creates a registered student account in Supabase. 2. Administrator retrieves student's UUID. 3. In Supabase SQL Editor, administrator executes: `SELECT public.bootstrap_first_admin('<TARGET_USER_UUID>');` 4. Function acquires 64-bit advisory lock `847291837461928374`. 5. Function verifies 0 administrators exist. 6. Function inserts authorization record into `private_ccms.admin_bootstrap_tokens`. 7. Function updates target user role to `'admin'`. 8. Trigger `protect_profile_role` validates token presence and permits role change. 9. Function deletes token and releases advisory lock on transaction commit.
- **Alternative Flow (Admin Already Exists):**
- Procedure raises `EXCEPTION: 'Bootstrap denied: The system already has one or more administrators configured.'`.
- **Postconditions:** Target user is verified `'admin'`; subsequent bootstrap executions are permanently blocked.

---

8. Requirement Traceability

8.1 Traceability Matrix

This matrix maps each Functional Requirement to its Architectural Component, Database Entities, and Verification Test Case IDs.

Req ID	Requirement Description	Architecture Module / Service	Database Entity / Policy / Trigger	Test Case ID
FR-AUTH-01	Student Self-Registration	`Register.tsx`, `supabase.auth`	`auth.users`, `handle_new_user()`	`TC-AUTH-01`
FR-AUTH-02	User Login & Authentication	`Login.tsx`, `AuthContext.tsx`	`auth.users`, `public.profiles`	`TC-AUTH-02`
FR-AUTH-03	Role-Based Route Guard	`ProtectedRoute.tsx`, `App.tsx`	`public.profiles.role`	`TC-AUTH-03`
FR-AUTH-04	Role Tamper Protection	`protect_profile_role()`	`profiles_update_policy`, trigger	`TC-AUTH-04`
FR-AUTH-05	First-Admin Bootstrap	`bootstrap_first_admin(uuid)`	`private_ccms.admin_bootstrap_tokens`	`TC-AUTH-05`
FR-AUTH-06	Secure Session Logout	`AuthContext.tsx`	Supabase GoTrue Auth Token	`TC-AUTH-06`
FR-CLUB-01	Public Club Directory	`Clubs.tsx`, `services/clubs.ts`	`public.clubs`, `clubs_select_policy`	`TC-CLUB-01`
FR-CLUB-02	Category Filtering	`Clubs.tsx`, `Landing.tsx`	`public.club_categories`	`TC-CLUB-02`
FR-CLUB-03	Instant Club Text Search	`Clubs.tsx`	`public.clubs`	`TC-CLUB-03`
FR-CLUB-04	Club Details Page	`ClubDetails.tsx`	`public.clubs.slug`	`TC-CLUB-04`

Req ID	Requirement Description	Architecture Module / Service	Database Entity / Policy / Trigger	Test Case ID
FR-CLUB-05	Admin Club Creation	`AdminDashboard.tsx`	`clubs_insert_policy`, `is_admin()`	`TC-CLUB-05`
FR-CLUB-06	Coordinator Club Editing	`CoordinatorDashboard.tsx`	`clubs_update_policy`	`TC-CLUB-06`
FR-MEM-01	Club Membership Application	`ClubDetails.tsx`	`public.club_memberships`	`TC-MEM-01`
FR-MEM-02	Duplicate Application Guard	Database constraint	`UNIQUE (club_id, student_id)`	`TC-MEM-02`
FR-MEM-03	Membership Immutability	Database trigger	`protect_club_membership_immutable_fields`	`TC-MEM-03`
FR-MEM-04	Coordinator Member Review	`CoordinatorDashboard.tsx`	`memberships_update_policy`	`TC-MEM-04`
FR-MEM-05	Membership Withdrawal	`StudentDashboard.tsx`	`memberships_delete_policy`	`TC-MEM-05`
FR-EVT-01	Coordinator Event Creation	`CoordinatorDashboard.tsx`	`public.events`, `events_insert_policy`	`TC-EVT-01`
FR-EVT-02	Public Event Browsing	`ClubDetails.tsx`, `Landing.tsx`	`events_select_policy`	`TC-EVT-02`
FR-EVT-03	Event Modification	`CoordinatorDashboard.tsx`	`events_update_policy`	`TC-EVT-03`
FR-EVT-04	Event Deletion Cascade	`CoordinatorDashboard.tsx`	`events_delete_policy`	`TC-EVT-04`
FR-REG-01	Event Registration	`ClubDetails.tsx`, `services/events.ts`	`public.event_registrations`	`TC-REG-01`
FR-REG-02	Duplicate Registration Guard	Database constraint	`UNIQUE (event_id, student_id)`	`TC-REG-02`
FR-REG-03	Capacity Cap Validation	`services/events.ts`	`events.capacity`, registration count	`TC-REG-03`
FR-REG-04	Registration Cancellation	`StudentDashboard.tsx`	`registrations_delete_policy`	`TC-REG-04`
FR-REG-05	Registration Roster Access	`CoordinatorDashboard.tsx`	`registrations_select_policy`	`TC-REG-05`
FR-ATT-01	Attendance Roster Fetch	`CoordinatorDashboard.tsx`	`public.attendance`, `attendance_select`	`TC-ATT-01`
FR-ATT-02	Attendance Marking	`CoordinatorDashboard.tsx`	`attendance_insert_policy`	`TC-ATT-02`
FR-ATT-03	Attendance Modification	`CoordinatorDashboard.tsx`	`attendance_update_policy`	`TC-ATT-03`
FR-ATT-04	Student Attendance View	`StudentDashboard.tsx`	`attendance_select_policy`	`TC-ATT-04`
FR-ATT-05	Student RLS Write Block	Database policy	`attendance_insert_policy` check	`TC-ATT-05`
FR-ANN-01	Announcement Creation	`CoordinatorDashboard.tsx`	`announcements_insert_policy`	`TC-ANN-01`
FR-ANN-02	Public Announcement Feed	`ClubDetails.tsx`	`announcements_select_policy`	`TC-ANN-02`

Req ID	Requirement Description	Architecture Module / Service	Database Entity / Policy / Trigger	Test Case ID
FR-ANN-03	Student Announcement Stream	`StudentDashboard.tsx`	`announcements_select_policy`	`TC-ANN-03`
FR-ANN-04	Announcement Deletion	`CoordinatorDashboard.tsx`	`announcements_delete_policy`	`TC-ANN-04`
FR-ADM-01	Coordinator Assignment	`AdminDashboard.tsx`	`public.club_coordinators`	`TC-ADM-01`
FR-ADM-02	Automatic Role Elevation	Database trigger	`sync_coordinator_role_on_assign`	`TC-ADM-02`
FR-ADM-03	Automatic Role Demotion	Database trigger	`sync_coordinator_role_on_unassign`	`TC-ADM-03`
FR-ADM-04	Club Category Management	`AdminDashboard.tsx`	`categories_insert_policy`	`TC-ADM-04`
FR-ADM-05	Visual Analytics Dashboard	`AdminDashboard.tsx` (Recharts)	Aggregated queries over clubs/events	`TC-ADM-05`
FR-ADM-06	User Directory Governance	`AdminDashboard.tsx`	`profiles_select_policy`, `is_admin()`	`TC-ADM-06`

9. Document References & Supporting Information

9.1 Applicable Standards

1. **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
2. **IEEE Std 1012-2016:** IEEE Standard for System, Software, and Hardware Verification and Validation.
3. **ISO/IEC/IEEE 29119:** Software and Systems Engineering — Software Testing standards.
4. **W3C Web Content Accessibility Guidelines (WCAG) 2.1:** Level AA compliance rules.
5. **RFC 7519:** JSON Web Token (JWT) architecture and verification specification.

9.2 Definitions, Acronyms, and Abbreviations

- **ACID:** Atomicity, Consistency, Isolation, Durability (database transaction properties).
- **CCMS:** College Club Management System.
- **CLS:** Cumulative Layout Shift (Web Vitals metric measuring visual stability).
- **FCP:** First Contentful Paint.
- **GUC:** Grand Unified Configuration (PostgreSQL configuration parameter setting).
- **HMR:** Hot Module Replacement (instant developer module reload via Vite).
- **IA3:** Internal Assessment 3 (Academic software engineering project milestone).
- **JWT:** JSON Web Token (RFC 7519 cryptographically signed bearer token).
- **LCP:** Largest Contentful Paint.
- **PL/pgSQL:** Procedural Language / PostgreSQL Structured Query Language.
- **RBAC:** Role-Based Access Control.
- **RLS:** Row Level Security (PostgreSQL security mechanism restricting row access by user context).
- **RTM:** Requirement Traceability Matrix.
- **SECURITY DEFINER:** A PostgreSQL function attribute specifying execution with the privileges of the function's creator rather than the invoking user.
- **SPA:** Single Page Application.

- **SRS:** Software Requirements Specification.
- **TLS:** Transport Layer Security (cryptographic communication security protocol).
- **UUID:** Universally Unique Identifier (128-bit value conforming to RFC 4122).
- **WCAG:** Web Content Accessibility Guidelines.

Document Approval Record

Role	Name	Signature / Status	Date
Lead Software Engineer	CCMS Project Team	Approved	November 1, 2026
Quality Assurance Lead	IA3 Evaluation Team	Verified	November 1, 2026
Course Instructor / Evaluator	Department of Software Engineering	Accepted	November 1, 2026